PRACTICAL 5-A

Object Oriented Programming

AIM:

To implement and understand the concepts specified in Practical 5 of the CEUE203 Object Oriented Programming practical list, using Java. The work covers Part A: Shape Areas, Payroll, Media Late Fees and Part B: Abstract Account subclasses and polymorphism.

CODE SCREENSHOT:



Code Screenshot 1

Payroll.java

Part A > Practical 5-A > Payroll.java > ...

```java
abstract class Employee {
    String name, id;

    Employee(String n, String i) {
        name = n;
        id = i;
    }

    abstract double monthlySalary();
}

class FullTime extends Employee {
    double salary;

    FullTime(String n, String i, double s) {
        super(n, i);
        salary = s;
    }

    double monthlySalary() {
        return salary;
    }
}

class PartTime extends Employee {
    int hours;
    double rate;

    PartTime(String n, String i, int h, double r) {
        super(n, i);
        hours = h;
        rate = r;
    }

    double monthlySalary() {
        return hours * rate;
    }
}

class Intern extends Employee {
    double stipend;

    Intern(String n, String i, double s) {
        super(n, i);
        stipend = s;
    }

    double monthlySalary() {
```

Code Screenshot 2

ShapeDemo.java

Part A > Practical 5-A > ShapeDemo.java > ...

```java
abstract class Shape {
    abstract double area();
}

class Circle extends Shape {
    double r;

    Circle(double r) {
        this.r = r;
    }

    double area() {
        return Math.PI * r * r;
    }
}

class Rectangle extends Shape {
    double w, h;

    Rectangle(double w, double h) {
        this.w = w;
        this.h = h;
    }

    double area() {
        return w * h;
    }
}

class Triangle extends Shape {
    double b, h;

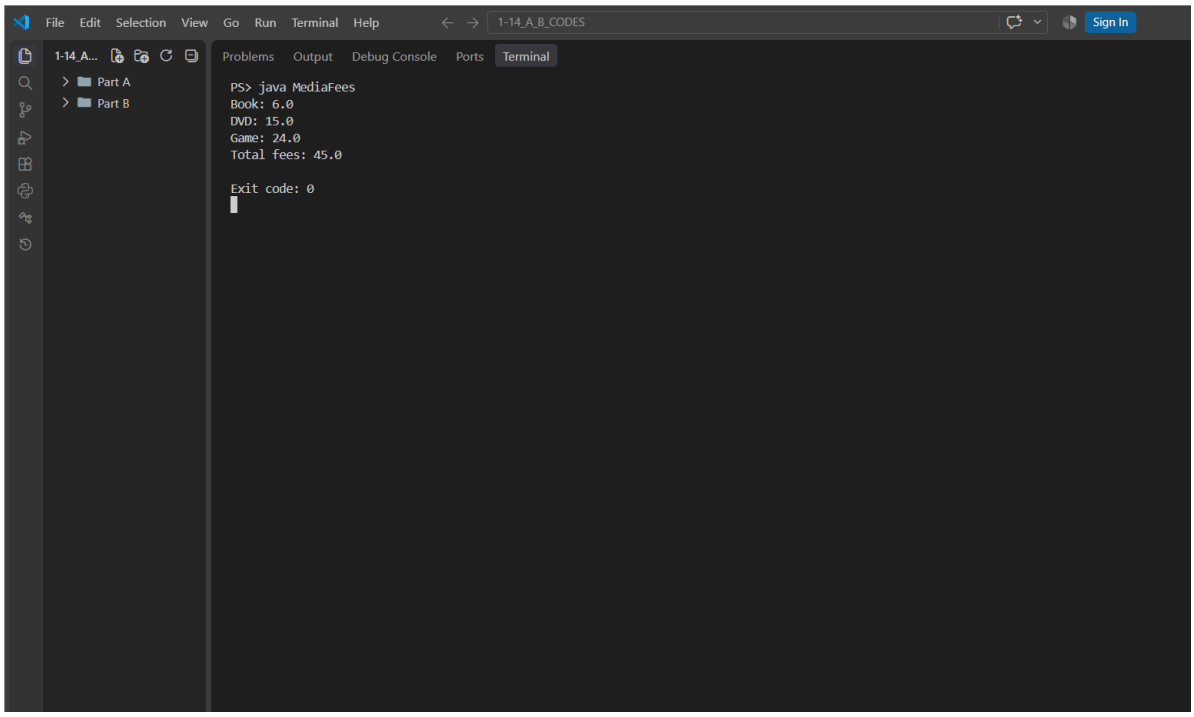
    Triangle(double b, double h) {
        this.b = b;
        this.h = h;
    }

    double area() {
        return b * h / 2;
    }
}

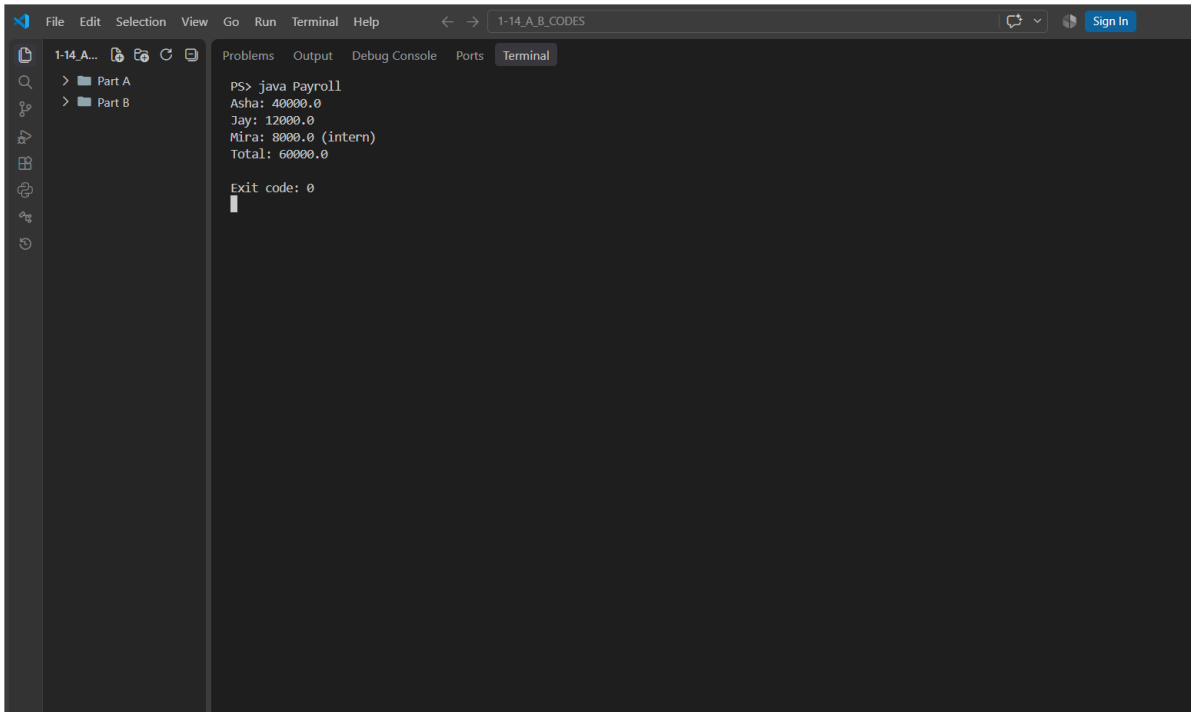
public class ShapeDemo {
    Run | Debug
    public static void main(String[] args) {
        Shape[] shapes = { new Circle(r: 2), new Rectangle(w: 3, h: 4), new Triangle(b: 3, h: 5) };
        double total = 0, largest = 0;
        for (Shape s : shapes) {
```

Code Screenshot 3

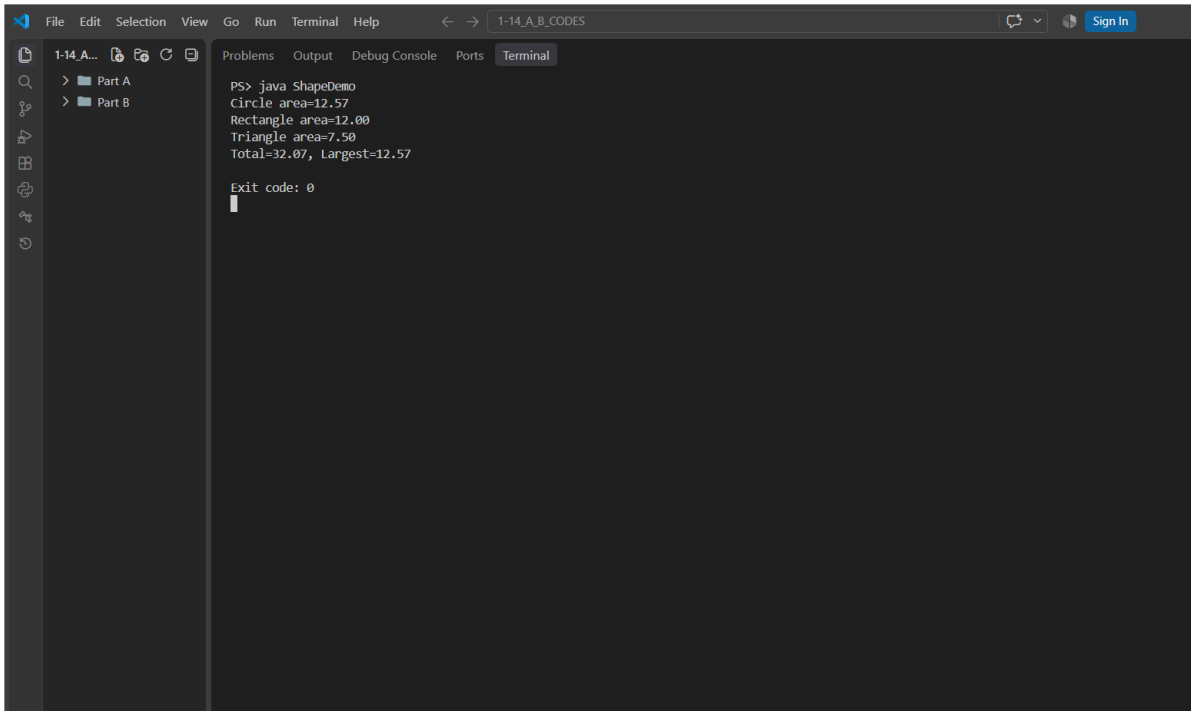
OUTPUT SCREENSHOT:



Output Screenshot 1



Output Screenshot 2



Output Screenshot 3

LEARNING OUTCOMES:

Use inheritance, abstract classes, method overriding, and runtime polymorphism to model related types.